

SECCIÓN DE RECURSOS - Documentación Completa

Índice

1. [Concepto y Propósito](#)
 2. [Estructura de Contenidos](#)
 3. [Arquitectura Técnica](#)
 4. [Flujos de Datos](#)
 5. [Implementación en Be Stronger](#)
 6. [Adaptación para tu App](#)
-

Concepto y Propósito

¿Qué es la Sección de Recursos?

La **Sección de Recursos** es un hub educativo centralizado dentro de la app donde los clientes acceden a:

- **Contenido educativo** personalizado según su plan
- **Guías prácticas** sobre técnica, nutrición, hábitos
- **Documentos descargables** (PDFs, planes nutricionales)
- **Vídeos tutoriales** (técnica de ejercicios, recetas)
- **Artículos y blogs** sobre entrenamiento y salud

Objetivos Principales

Objetivo	Descripción
Educación	Enseñar al cliente cómo ejecutar su plan
Autonomía	Que el cliente sea independiente sin preguntar
Retención	Mantener al cliente enganchado con contenido valioso
Diferenciación	Ofrecer valor más allá del entrenamiento

Escalabilidad

Crear contenido una vez, distribuir a muchos clientes

Valor para el Negocio

Plain Text

```
Cliente recibe contenido valioso
  ↓
Cliente se siente apoyado
  ↓
Cliente progresa más rápido
  ↓
Cliente renueva su membresía
  ↓
Recomendaciones a amigos
  ↓
Crecimiento exponencial
```

Estructura de Contenidos

Categorías Principales

Plain Text

RECURSOS

-  EDUCACIÓN
 - Guía de Técnica
 - Guía de Nutrición
 - Guía de Recuperación
 - Guía de Mentalidad
-  VÍDEOS
 - Técnica de Ejercicios
 - Recetas Saludables
 - Rutinas de Movilidad
 - Testimonios
-  DOCUMENTOS
 - Plan Nutricional (PDF)
 - Mesociclo Detallado (PDF)
 - Recetario (PDF)

└─	Checklist de Hábitos
└─	 SEGUIMIENTO
└─	Progreso de Cargas
└─	Evolución Corporal
└─	Hábitos Completados
└─	Estadísticas Personales
└─	 COMUNIDAD
└─	Preguntas Frecuentes
└─	Testimonios de Clientes
└─	Casos de Éxito
└─	Contacto Directo

Contenido por Tipo

1. Guías Educativas (HTML/Markdown)

- Onboarding: Cómo empezar
- Técnica: Cómo hacer cada ejercicio
- Nutrición: Cómo estructurar comidas
- Recuperación: Deload, descanso, movilidad
- Mentalidad: Consistencia, motivación

2. Vídeos (YouTube/Vimeo)

- Demostraciones de técnica
- Recetas paso a paso
- Sesiones de Q&A
- Transformaciones antes/después

3. Documentos Descargables (PDF)

- Plan nutricional personalizado
- Mesociclo de 4 semanas
- Recetario con macros
- Manual de autogestión

4. Datos de Seguimiento (Gráficos)

- Progresión de cargas semana a semana

- Evolución de peso/medidas
- Cumplimiento de hábitos
- Comparativa antes/después

Arquitectura Técnica

Stack Tecnológico

Plain Text

CLIENTE (React/React Native)

Componentes UI

- ResourcesList.tsx
- ResourceDetail.tsx
- DocumentViewer.tsx
- VideoPlayer.tsx
- ProgressChart.tsx

↓ (API REST/tRPC)

BACKEND (Node.js/Express)

API Endpoints

- GET /resources (lista de recursos)
- GET /resources/:id (detalle)
- GET /documents/:id (descarga PDF)
- GET /progress/:clientId (gráficos)

Lógica de Negocio

- Filtrar recursos por cliente
- Generar documentos dinámicos
- Calcular progresión
- Personalizar contenido

↓ (SQL)

BASE DE DATOS

Tablas

- resources (contenido)
- client_resources (asignación)
- resource_views (tracking)
- documents (PDFs)
- progress (seguimiento)

↓ (S3/Cloud Storage)

ALMACENAMIENTO

- Vídeos (YouTube/Vimeo)
- PDFs (S3)
- Imágenes (CDN)
- Documentos dinámicos

Modelo de Base de Datos

SQL

-- Tabla de recursos disponibles

```
CREATE TABLE resources (
  id INT PRIMARY KEY,
  title VARCHAR(255),
  description TEXT,
  category ENUM('guide', 'video', 'document', 'tracking'),
  type ENUM('html', 'pdf', 'youtube', 'chart'),
  content_url VARCHAR(500),
  created_at TIMESTAMP,
  updated_at TIMESTAMP
);
```

-- Asignación de recursos a clientes

```
CREATE TABLE client_resources (
  id INT PRIMARY KEY,
  client_id INT,
  resource_id INT,
  assigned_at TIMESTAMP,
  access_level ENUM('view', 'download', 'edit'),
  FOREIGN KEY (client_id) REFERENCES clients(id),
  FOREIGN KEY (resource_id) REFERENCES resources(id)
);
```

-- Seguimiento de visualizaciones

```
CREATE TABLE resource_views (
  id INT PRIMARY KEY,
  client_id INT,
```

```

    resource_id INT,
    viewed_at TIMESTAMP,
    time_spent INT, -- segundos
    completed BOOLEAN
);

-- Documentos personalizados
CREATE TABLE documents (
    id INT PRIMARY KEY,
    client_id INT,
    type ENUM('nutrition_plan', 'mesociclo', 'recipes'),
    file_url VARCHAR(500),
    generated_at TIMESTAMP,
    expires_at TIMESTAMP
);

-- Datos de progresión
CREATE TABLE progress (
    id INT PRIMARY KEY,
    client_id INT,
    metric_type ENUM('weight', 'strength', 'habits'),
    value FLOAT,
    recorded_at TIMESTAMP,
    week INT
);

```

Flujos de Datos

Flujo 1: Cliente Accede a Recursos

Plain Text

```

1. Cliente abre app
  └─ Se autentica con OAuth/JWT

```

↓

```

2. App solicita recursos
  └─ GET /api/resources?clientId=123

```

↓

```

3. Backend procesa solicitud
  └─ Valida que cliente existe

```

- └─ Obtiene recursos asignados
- └─ Filtra por plan/nivel
- └─ Retorna JSON con lista

↓

4. App renderiza lista
 - └─ Muestra categorías
 - └─ Muestra recursos con thumbnails
 - └─ Permite buscar/filtrar

↓

5. Cliente hace clic en recurso
 - └─ App abre detalle del recurso

Flujo 2: Cliente Descarga Documento Personalizado

Plain Text

1. Cliente solicita documento
 - └─ GET /api/documents/nutrition-plan

↓

2. Backend verifica si existe
 - └─ ¿Existe en BD? → Retornar URL
 - └─ ¿No existe? → Generar nuevo

↓

3. Si no existe, generar dinámicamente
 - └─ Obtener datos del cliente
 - └─ Peso, objetivo, alergias, preferencias
 - └─ Obtener plan nutricional
 - └─ TDEE, macros, comidas
 - └─ Generar PDF
 - └─ Con datos personalizados
 - └─ Guardar en S3

↓

4. Retornar URL descargable

└ Cliente descarga PDF

Flujo 3: Mostrar Progresión del Cliente

Plain Text

1. Cliente abre sección "Mi Progreso"
 - └ GET /api/progress?clientId=123&weeks=12

↓

2. Backend obtiene datos
 - └ Cargas levantadas (semana 1-12)
 - └ Peso corporal (semana 1-12)
 - └ Hábitos completados (%)
 - └ Medidas corporales

↓

3. Procesar datos para gráficos
 - └ Calcular tendencia
 - └ Calcular promedio
 - └ Detectar anomalías
 - └ Generar JSON para Chart.js/Recharts

↓

4. App renderiza gráficos
 - └ Gráfico de línea: Progresión de cargas
 - └ Gráfico de área: Evolución de peso
 - └ Gráfico de barras: Hábitos completados
 - └ Comparativa antes/después



Implementación en Be Stronger

Cómo lo Construimos

1. Estructura de Carpetas

Plain Text


```

coaching-powerlifting/
├── server/
│   ├── guides/                                # Guías HTML
│   │   ├── onboarding.html
│   │   ├── tecnica-levantamiento.html
│   │   ├── nutricion.html
│   │   └── ...
│   ├── clientsRouter.ts                       # Endpoints para clientes
│   ├── clientsDb.ts                           # Funciones BD
│   ├── nutritionDb.ts                         # Funciones nutrición
│   └── guidesRouter.ts                       # Endpoints de guías
├── client/
│   ├── src/pages/
│   │   ├── ClientDashboard.tsx               # Panel principal
│   │   ├── ClientMesociclo.tsx              # Mesociclo
│   │   └── ClientResources.tsx              # Recursos (no implementado)
│   └── src/components/
│       └── ResourceCard.tsx                  # Componente reutilizable
└── drizzle/
    └── schema.ts                             # Tablas BD

```

2. Endpoints API Creados

TypeScript

```

// GET /api/trpc/clientAuth.getMesocicloContent
// Obtiene el HTML del mesociclo para mostrar en iframe

// GET /api/trpc/clientAuth.getGuides
// Obtiene lista de guías disponibles

// GET /api/trpc/clientAuth.getNutritionPlan
// Obtiene plan nutricional personalizado

// GET /api/trpc/clientAuth.getProgress
// Obtiene datos de progresión para gráficos

```

3. Componentes React Creados

TypeScript

```

// ClientDashboard.tsx
- Muestra lista de recursos disponibles

```

- Tarjetas con iconos y títulos
- Navegación entre secciones

// ClientMesocicloPage.tsx

- Carga mesociclo en iframe
- Muestra contenido **HTML**

// ResourceCard.tsx

- Componente reutilizable
- Muestra thumbnail, título, descripción
- Botón de acceso

4. Base de Datos

TypeScript

// Tabla clients

- mesocicloActual: **número** (**1-4**)
- guidesAccess: boolean
- nutritionAccess: boolean

// Tabla client_nutrition

- mesociclo1Url: **string** (**URL** a **HTML** en **S3**)
- mesociclo2Url: string
- mesociclo3Url: string
- mesociclo4Url: string

// Tabla guides (no creada aún)

- id: int
- title: string
- slug: string
- content: **text** (**HTML**)
- category: enum

5. Archivos Estáticos

Plain Text

/manus-storage/

- └─ mesociclo-m1-mac2_cd600dd0.html
- └─ mesociclo-m2-mac2_cd600dd0.html
- └─ guia-tecnica.html
- └─ guia-nutricion.html
- └─ recetario.pdf

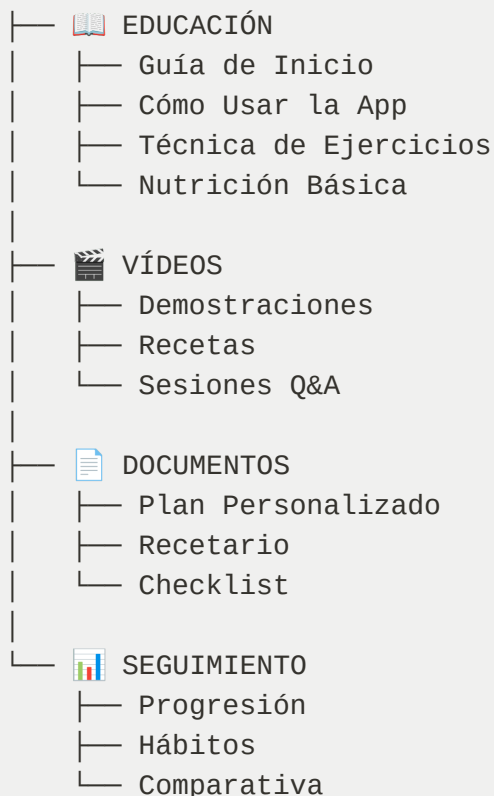
Adaptación para tu App

Paso 1: Diseñar Estructura de Recursos

Define qué recursos necesita tu app:

Plain Text

TU APP - RECURSOS



Paso 2: Crear Tablas en BD

SQL

```
-- Recursos disponibles
CREATE TABLE resources (
  id INT PRIMARY KEY AUTO_INCREMENT,
  title VARCHAR(255) NOT NULL,
  description TEXT,
  category VARCHAR(50),
  type VARCHAR(50),
  content_url VARCHAR(500),
  thumbnail_url VARCHAR(500),
  order_index INT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);

-- Asignación a clientes
CREATE TABLE client_resources (
  id INT PRIMARY KEY AUTO_INCREMENT,
  client_id INT NOT NULL,
  resource_id INT NOT NULL,
  assigned_date TIMESTAMP,
  viewed BOOLEAN DEFAULT FALSE,
  viewed_date TIMESTAMP,
  FOREIGN KEY (client_id) REFERENCES clients(id),
  FOREIGN KEY (resource_id) REFERENCES resources(id)
);
```

Paso 3: Crear Endpoints Backend

TypeScript

```
// Express.js ejemplo

// Obtener recursos de un cliente
app.get('/api/resources/:clientId', async (req, res) => {
  const { clientId } = req.params;

  // Validar cliente
  const client = await db.clients.findById(clientId);
  if (!client) return res.status(404).json({ error: 'Cliente no encontrado' });

  // Obtener recursos asignados
  const resources = await db.query(`
    SELECT r.* FROM resources r
    JOIN client_resources cr ON r.id = cr.resource_id
    WHERE cr.client_id = ?
    ORDER BY r.order_index
  `, [clientId]);

  res.json(resources);
});

// Obtener detalle de recurso
app.get('/api/resources/:clientId/:resourceId', async (req, res) => {
  const { clientId, resourceId } = req.params;

  // Validar acceso
  const hasAccess = await db.query(`
    SELECT 1 FROM client_resources
    WHERE client_id = ? AND resource_id = ?
  `);
```

```

`, [clientId, resourceId]);

if (!hasAccess) return res.status(403).json({ error: 'Sin acceso' });

// Obtener recurso
const resource = await db.resources.findById(resourceId);

// Registrar visualización
await db.query(`
  UPDATE client_resources
  SET viewed = TRUE, viewed_date = NOW()
  WHERE client_id = ? AND resource_id = ?
`, [clientId, resourceId]);

res.json(resource);
});

// Generar documento personalizado
app.get('/api/documents/:clientId/nutrition-plan', async (req, res) => {
  const { clientId } = req.params;

  // Obtener datos del cliente
  const client = await db.clients.findById(clientId);
  const nutrition = await db.nutrition.findByClientId(clientId);

  // Generar PDF dinámico
  const pdf = await generateNutritionPDF(client, nutrition);

  // Guardar en S3
  const url = await s3.upload(pdf, `nutrition-${clientId}.pdf`);

  res.json({ url });
});

```

Paso 4: Crear Componentes React Native

TypeScript

```

// ResourcesScreen.tsx
import React, { useEffect, useState } from 'react';
import { View, FlatList, TouchableOpacity, Text } from 'react-native';

export default function ResourcesScreen({ clientId }) {
  const [resources, setResources] = useState([]);

  useEffect(() => {
    fetchResources();
  });

```

```

}, []));

const fetchResources = async () => {
  const response = await fetch(`/api/resources/${clientId}`);
  const data = await response.json();
  setResources(data);
};

const handleResourcePress = (resource) => {
  // Navegar a detalle
  navigation.navigate('ResourceDetail', { resource });
};

return (
  <View>
    <FlatList
      data={resources}
      renderItem={({ item }) => (
        <TouchableOpacity onPress={() => handleResourcePress(item)}>
          <ResourceCard resource={item} />
        </TouchableOpacity>
      )}
      keyExtractor={item => item.id.toString()}
    />
  </View>
);
}

// ResourceDetailScreen.tsx
export default function ResourceDetailScreen({ route }) {
  const { resource } = route.params;

  if (resource.type === 'html') {
    return <WebView source={{ uri: resource.content_url }} />;
  }

  if (resource.type === 'pdf') {
    return <PDFView source={resource.content_url} />;
  }

  if (resource.type === 'video') {
    return <VideoPlayer source={resource.content_url} />;
  }
}

```

Checklist de Implementación

- ☐ Diseñar estructura de recursos
 - ☐ Crear tablas en BD
 - ☐ Crear endpoints API
 - ☐ Crear componentes React/React Native
 - ☐ Implementar autenticación
 - ☐ Implementar búsqueda/filtros
 - ☐ Crear sistema de tracking
 - ☐ Implementar generación de documentos
 - ☐ Crear gráficos de progresión
 - ☐ Testing
 - ☐ Documentación
-

Conclusión

La **Sección de Recursos** es un componente crítico que:

- ☒ Educa al cliente
- ☒ Mejora retención
- ☒ Diferencia tu producto
- ☒ Escala sin costo marginal
- ☒ Genera datos valiosos

Con esta arquitectura, puedes construir un sistema robusto, escalable y personalizado para tus clientes.